



Sandya Mannarswamy

Welcome to another instalment of CodeSport. In this month's column, we continue our discussion on the false sharing issue in multi-threaded applications. We'll also discuss priority inversion and possible techniques to deal with it.

Thanks to all the readers who responded to the problems we discussed in last month's column. We had given a small code snippet of multi-threaded code, which exhibits false sharing of data. We had asked readers to rewrite the code to avoid the false sharing issue. Congratulations to Nilesh Govande, Arivendu Bhardwaj, Vivek Goel and Miranda Walters for getting the correct answer. Here is the original buggy code snippet from the takeaway problem:

```
int num_tickets_per_thread [N]; //N is the number of
threads in your application

IncrementTicketCounter()
{
    int thread_id = pthread_self();
    num_tickets_per_thread[thread_id]++;
}

void PrintTotalTickets()
{
    //only the main thread calls this function, so no
    locking needed
    for (int i = 0; i < N; i++)
        num_total_tickets += Num_tickets_per_thread[i];
    printf ("%d", num_total_tickets);
}
```

Assuming a cache line of size 64 bytes, we see that 16 elements of the array `num_tickets_per_thread` can fit in the same cache line. Recall that a processor updating the array element `num_tickets_`

`per_thread[thread_id]` ends up invalidating the other processor's cache copies because of their co-location in the same cache line. Unlike true sharing where the same datum is accessed by multiple threads, in this case, different threads access different data items. Because of their co-location in the same cache line, the line gets migrated across the different processor caches. This phenomenon is known as false sharing. This results in an increased number of cache misses and will show up as an increase in the execution time of the application.

```
#define CACHE_LINE_SIZE 64 //we assume a 64 byte
cache line size
struct ticket
{
    int number;
    int dummy_pad[CACHE_LINE_SIZE - sizeof(int)];
};

struct ticket num_tickets_per_thread [N];
//N is the number of threads in your application

IncrementTicketCounter()
{
    int thread_id = pthread_self();
    num_tickets_per_thread[thread_id].number++;
}

void PrintTotalTickets()
{
    //only the main thread calls this function, so no
    locking needed
    for (int i = 0; i < N; i++)
```